BADNET: Enhancing Resistance to Censorship of Directory Authority in Anonymity Networks with Ethereum Smart Contracts

Anonymous Author(s)

Abstract

Anonymity network is a privacy-enhancing tool that enables users to explore the Internet without revealing their IP address. State-of-the-art anonymity networks typically employ centralized directory authorities (DirAuths) that are vulnerable to blocking and censorship. Particularly in anonymity networks with relatively few participants, an adversary can simply disable the network by preventing connections to the DirAuths, or track users by monitoring network traffic to the DirAuths. This paper presents BADNET, a novel anonymity network architecture that combines blockchain and onion routing to evade censorship, allowing the DirAuths to be hidden behind a decentralized blockchain system. A key idea of BADNET is to use Ethereum smart contracts to take over the core functions of the directory service so that relays and clients are no longer directly connected to DirAuths. BADNET adopts a broadcast encryption algorithm and a new circuit construction protocol to further enhance its resistance to censorship. We implement a prototype system using the Ropsten Ethereum testnet that provides over 10,000 anonymous communication services in one month, demonstrating the practicality of BADNET in real scenarios.

1 Introduction

Due to numerous data surveillance, public concern over privacy has been likely at one of its peaks. NSA builds a surveillance network that covers roughly 75% of all US Internet traffic in the hunt for so-called foreign intelligence [24]. Field studies have manifested that anonymity networks serve privacy protection in Internet applications [55]. Anonymous communication protocol (ACP) formalizes efforts to address communication surveillance issues by routing techniques. Kinds of ACPs (e.g., Mixes [11], Onion Routing [18, 50], Crowds [51]) have been proposed for different requirements, where they are divided into high-latency networks with strong anonymity, and low-latency networks with better performance. One of the most widely known ACPs is Tor [18], as it balances anonymity with good end-to-end performance, especially in terms of network latency. To enable automatic fetching of up-to-date network status while ensuring trust of obtained information, Tor adopts a set of centralized *Directory Authorities* (DirAuths) to provide *directory services* that collect and distribute relay information. State-of-the-art ACPs mostly rely on the centralized directory authority for relay discovery.

However, Tor's DirAuths is a vulnerability of the network as they have public IP addresses and frequently communicate with relays and clients. Once an adversary subverts or destroys DirAuths via fixed IPs, it will incur the anonymity network insecurity or even paralysis. Tor strengthens its resistance to censorship by expanding the number of DirAuths to nine, while introducing fallback directory mirrors. Unfortunately, a resourceful adversary can prevent access to anonymity networks by blocking connections to all DirAuths and relays.

This issue is more pronounced in small-scale anonymity networks. With millions of users from all over the world, Tor can hide the traffic of a certain user, ensuring that an adversary cannot know which websites the user has visited. However, Tor does not focus on hiding the fact that users are using an anonymity network to browse the Internet. Some sensitive or censored users are unwilling or unable to connect to public Tor relays and turn to alternative Tor protocol-based anonymity networks. Such anonymity networks have relatively limited number of DirAuths and relays, making them more vulnerable to surveillance and blocking. More importantly, users of such anonymity networks usually come from a specific group, so an adversary might discover the identity of users once the IP of DirAuths is exposed.

At a glance, the above issues can be trivially solved by removing the DirAuths. Completely decentralized P2P anonymity networks [1, 22, 45, 52] successfully eliminate the central infrastructure while making the network more scalable since every client acts as a relay. However, the absence of central control complicates relay discovery, due to some inherent problems, i.e., the dynamic nature of peer participation, and a variety of attacks based on nodes having imperfect knowledge of the set of relays [9, 14, 15, 57]. Furthermore, many nodes

may be unable or unwilling to participate as relays, so how to break aforesaid limitations while keeping a client-server architecture seems to be yet challenging. Blockchain is a decentralized network capable of providing trust that does not rely on a central node. Blockchain technology has been applied in many fields, such as blockchain-based financial system, blockchain-based copyright depository system, etc. More importantly, blockchain is a type of distributed database that copies transaction data to a large number of peer nodes. Using blockchain transactions as rendezvous enables relay discovery without direct connection to dedicated nodes (e.g., normal relays and DirAuths), making directory services harder to be censored or blocked. Smart contracts implement complex functions on blockchain, while inheriting the characteristics of decentralization, enhanced security, and strong stability of the blockchain network. Therefore, a decentralized anonymity network architecture is possible if a Ethereum smart contract is used as the spokesman of the DirAuths.

Following the above design principles, we present BADNET, *a blockchain-based anonymity network architecture*. The key to this achievement is a principled hiding of the central authority behind a decentralized blockchain system, ensuring that the directory service in BADNET is resistant to DoS attacks, traffic eavesdropping attacks, etc. To support a secure and trusted decentralized directory service, we adopt an architecture where data protection is separated from smart contract. The relay adopts cryptographic algorithms for data confidentiality while the smart contract is responsible for curating and distributing relay information. In addition, considering the need to not expose relays, BADNET is built on top of a novel circuit construction protocol that require clients to maintain only the status information of relays, while keeping secure against epistemic and route capture attacks. Anchored in such a syncretic framework, BADNET's design can strip DirAuths of all direct connections to relay-and-client while providing directory data for relays and clients, respectively.

Technique Overview. BADNET utilizes a public blockchain system as a decentralized medium between relays, clients and DirAuths. In other words, the collection and distribution of relay information is handled by smart contracts, and the specific relay data is stored in the blockchain ledger. Therefore, BADNET's directory services are executed automatically by smart contract code rather than running on several dedicated servers. In addition, we introduce a broadcast encryption system in the relay group to share dynamic keys used to protect the relay data among BADNET relays. Neither relays that are not in the group (i.e., not registered) nor relays that have been removed from the group can successfully decrypt the encrypted content. So even if a malicious relay is detected, relays do not must renegotiate keys among them to prevent further relay exposure.

Contributions. In this paper, we explore how the blockchain seamlessly works with anonymity networks. The exploration spans both theoretical algorithms and practical prototype perspectives, and is specifically driven by the vulnerable issue of exposed (and centralized) authorities in dedicated anonymity networks (for example, a system deployed to evade Internet censorship). We make three contributions:

- We propose a new anonymity network architecture based on blockchain and onion routing, which hides directory authorities behind a blockchain system and provides directory services via smart contracts, which is particularly suitable for anonymity networks with few participants.
- We design a circuit construction protocol, which enables secure path selection without exposing all relays, reducing the risk of IP exposure for relays.
- We design a broadcast encryption-based data protection strategy that allows only approved relays to decrypt data read from the blockchain and provide services, and can remove compromised or expired relays at low cost.

We implement a proof-of-concept prototype that realizes BADNET for anonymous communication applications. The implementation includes 32 relays throughout the world, which had been operating stably for one month, providing a total of over 10,000 anonymous communication services. Statistics show that 90% of anonymous communication services have a latency of less than 3s, which is comparable to the public Tor. To test the practicality of BADNET in real scenarios, we conduct evaluations on the Ropsten testnet using different sizes of relays. When simulating a full of Tor's scale, BADNET generated 6,912 blockchain transactions per day during its operation, accounting for about 11.5% of the total daily transaction, with an average transaction latency of 103.5s. We conclude that the BADNET solution is able to meet the performance requirements of anonymity network in real scenarios. Our data and code can be found at: https://github.com/Decentralized-Anonymity-Network/BadNet.

2 Background

2.1 Tor Network

Tor [18] is a low-latency anonymity network that consists of roughly 6,500 volunteer-run relays (i.e., onion routers) and two to eight million daily users [37, 59]. These users rely on Tor to anonymize their Internet connections, and also to evade censorship and surveillance. To provide sender anonymity, Tor forwards Internet traffic through source-routed paths constructed by (typically) three relays called circuits, i.e., relays forming the anonymous route set are selected by the client.

Tor circuits use multiple layers (three layers by default) of encryption to hide the source and destination of communications. Encrypted traffic typically enters the network at a *guard* relay. Guards are relays that are fast and stable compared to other relays. Such entry guard mechanism reduces the likelihood of the client connecting to an attacker. The second hop

in a circuit is the *middle* relay that carries traffic to the *exit* relay that acts as the egress point of Tor. Each hop in a circuit peels away one layer of encryption to reveal the next hop.

Tor's directory system. To distribute the relays used to build the circuits, Tor introduces nine dedicated servers called directory authorities (DirAuths) to maintain the network status. The IP addresses of these DirAuths are provided with the Tor software. Every relay periodically uploads a *router descriptor* to all DirAuths, describing its IP, keys, capabilities, etc. The DirAuths collaborate hourly to vote and generate a consensus document listing all active relays in the current network, and then every participant downloads this consensus.

Tor design has several advantages over fully P2P architectures without a central authority. For example, it is difficult for adversaries to execute eclipse attacks [56]. However, the reliance on central authorities makes them an attractive target for attackers [41], and Tor's relay discovery has scalability issues. DirAuths are operated by the Tor project maintainers, and users must trust that these servers will not collude [45]. In contrast, highly decentralized anonymity networks hold the promise of better distribution of trust.

Several designs for scalable Tor anonymity network have been proposed. McLachlan et al. proposed *Torsk* [38] in which relay selection and directory service of Tor are replaced by a DHT design. To mitigate information leak attacks [42], Torsk uses secret buddy nodes to perform the lookup process. However, Wang et al. [61] demonstrated some vulnerabilities in the mechanism for obtaining secret buddies. *PIR-Tor* [43], proposed by Mittal et al., uses private information retrieval (PIR) techniques [3, 23] to securely distribute information about only a few relays to clients. PIR ensures that no one but the client itself knows which relays it has fetched. However, a PIR solution will inevitably increase computational overhead and thus session latency, and computational PIR has shown to be susceptible to attacks [34]. The design of *ConsenSGX* [54], introduced by Sasy et al., is similar to PIR-Tor, but it combines oblivious RAMs (ORAMs) and trusted execution environments (TEE) to provide a more efficient directory service. However, relying on trust in Intel SGX raises the risk of compromise [10]. Unlike the above schemes, *Walking Onions* [33] allows clients to obliviously choose random paths without any pre-fetched relay descriptors, yet can verify the correctness of their paths after they are constructed.

Load balancing of Tor. Tor relays currently transfer more than 250 Gbit/s in aggregate traffic from between millions of daily active clients. In order to balance such large traffic load across thousands of relays, while avoiding complete reliance on self-reported measurements [6], Tor introduces a load-balancing system. Nowadays, 6 of the 9 DirAuths run the load-balancing system as *Bandwidth Authorities* (BWAuths) to measure the relative performance of relays. But Jansen et al. [28] has shown that Tor's centralized load-balancing design is vulnerable to DoS attacks.

SmarTor [4] decentralizes BWAuths using Ethereum smart contracts and TEE, to reduce trust assumptions and increase resilience against resourceful adversaries. *PeerFlow* [29] proposes a peer-measurement process that uses relay observations of each other to determine relay bandwidths. More concretely, Tor relays periodically report the total number of bytes exchanged among them to DirAuths, which are responsible for aggregating the traffic data to produce weights.

Censorship circumvention of Tor. The IP addresses of all relays (including DirAuths) are publicly available, subjecting them to trivial analysis and blocking. Tor resists censors that block Tor traffic by using *pluggable transports* (PTs) and *bridges* [60]. PTs make Tor traffic appear as something else by disguising it, making it more difficult to identify. Bridges are secret Tor relays that are not published in the consensus. However, default bridges are also vulnerable to censorship due to the reliance on hard-coding IPs [36].

Decoy routing [31] is a solution to circumvent blocking by using decoy routers outside censored areas to effectively route traffic. Decoy routers act as said man-in-the-middle, forwarding requests to blocked sites and sending the contents back to the client. *Geographical avoidance* [32, 35] is another way of censorship circumvention, since it enables established Tor circuits to not traverse untrusted geographic regions.

Denial-of-service attacks against Tor. Previous works have shown the susceptibility of the Tor network to DoS [5, 20, 27, 28]. Most importantly, *sniper attack* is a memory-based DoS attack that can disable arbitrary Tor relays [27]. Simulating on Shadow [26], they demonstrate that an attacker can target the group of DirAuths and thus the network will not operate over time. Defenses against sniper attacks has been rolled out by the Tor Project [53]. However, this defense was not triggered in *Bandwidth-based DoS Attacks* described in [28], because they aim to consume bandwidth and do not require long queues. This work also shows bandwidth DoS attacks against particularly vulnerable Tor components, including Tor bridges and BWAuths, by flooding them with packets.

2.2 Blockchain

A blockchain is essentially a distributed ledger that consists of a continuously growing set of records. These records, called *transactions*, are organized in *blocks* that are linked together using cryptographic hashes. Blockchain employs consensus algorithms to maintain the sanctity of the data recorded on blocks. Data integrity is easily verified by signatures and hashes. Thus, storing data in blockchains provides trusted data sharing without relying on third parties, thereby reducing trust costs and increasing interaction efficiency.

Ethereum and smart contract. Ethereum [62] is one of the most actively used blockchains as it allows users to write decentralized applications using *smart contracts*, which are the core value of Ethereum. Smart contracts were introduced early to describe an automated contractual relationship, however, the concept remained elusive and obscure until the advent

of blockchain technology. Now, the most popular blockchain platform for executing smart contracts is Ethereum and *Solidity* is the frequently used language for writing Ethereum smart contracts. *Ethereum Virtual Machine* (EVM) is the environment in which smart contracts run on Ethereum.

To ensure fair compensation for expended mining effort, Ethereum pays *miners* some transaction fees proportional to the required computation. The fee refers to the *gas* required to successfully conduct a transaction. Ethereum has its own cryptocurrency, called Ether or ETH. The smallest base unit of ETH is *wei* ($1\ \mathsf{ETH} = 10^{18}$ wei), and *gwei* is the most common unit of ETH ($1\ \mathsf{ETH} = 10^{9}$ gwei) as gas prices are easily specified in gwei. An Ethereum account is an entity with an ETH balance that can create transactions and send them to the Ethereum blockchain. There are two types of accounts supported by Ethereum, *user accounts* (aka externally-owned accounts) and *smart contract accounts*.

Blockchain-based censorship resistant. Blockchains can serve as a cornerstone for overcoming censorship policies, as no single entity controls the ledger. Namecoin [2], one of the first forks of Bitcoin, is a decentralized DNS that aims to put an end to Internet censorship. Tithonus [49] is a censorship resistant system based on the Bitcoin network and blockchain, which camouflages client requests for sensitive content into inconspicuous Bitcoin transactions. Minaei et al. [39] presented employing blockchain transactions as rendezvous to bootstrap censorship resistant communications. That solution extends to 4 blockchains, including Bitcoin, Zcash, Monero, and Ethereum. Leveraging IPFS and Ethereum smart contracts, Xu et al. [63] hoped to ensure the accessibility of social media networks by making them more resistant to censorship.

3 Overview

3.1 High Level Design

BADNET retains the benefits of Tor's client-server architecture, but hides the DirAuths behind a decentralized blockchain system and strips it of all direct connections to relays and clients, increasing the difficulty of censoring the network. Figure 1 shows the high-level setting of BADNET, consisting of the following components.

- A *client* is a privacy-conscious user who seeks to utilize an anonymity network that increases her privacy. Clients not only hide their identities from network services, but they are also not associated with the DirAuths.
- The *network service* is the destination that a client wants to access via the anonymity network. Such services often have censorship risks (such as being monitored or blocked) without the use of anonymity networks.
- A *relay* refers to a trusted onion router that receives and transmits packets to another relay or the network service. Exit relays are more likely to be monitored, tracked or even controlled as they communicate directly with the network service.
- The *DirAuths* are the governors responsible for curating the network, including running trusted relays, collecting registration requests, assigning private keys and updating groups. DirAuths are trustworthy, but there is a risk of compromising user privacy if DirAuths are censored.
- The *smart contract* (SC) is created by the DirAuths on a public blockchain, which is responsible for automatically providing directory services, i.e., relay registration management and information collection and distribution.
- A *public blockchain* refers to Ethereum, which provides distributed data storage with the blockchain transaction form. More concretely, approved relays store registration data and their advertisement.

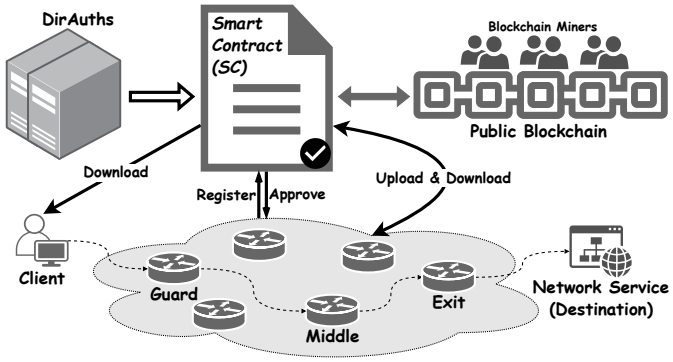


Figure 1: The architecture of BADNet. We adopt a model that separates capabilities from storage, using smart contracts for functionality and a public blockchain for data storage.

Our solution is designed for dedicated anonymity networks with a relatively small number of participants. Notably, such anonymity networks require additional security attributes compared to the classical Tor network and focus on the need for censorship resistance. Firstly, BADNET has to ensure that directory services are difficult to block by censors. Secondly, we argue that protection of the IPs of BADNET's DirAuths is necessary. Specifically, an adversary should not be able to discover connections to DirAuths in order to avoid DirAuths becoming a vulnerability. Finally, the IP of relays should be kept secret to prevent censors from tracking the identity of clients by eavesdropping on the relays.

3.2 Threat Model

Firstly, we consider a dishonest DirAuth. BADNET DirAuths have a governing capability over the network and their goal is to provide anonymity services to their user group. We argue

that DirAuths are inherently trustworthy, but also at risk of being compromised. We assume that the control of DirAuths over BADNET is dispersed. For example, if the DirAuths is owned by a single company with different employees responsible for SC creation, running relays and network maintenance, an adversary cannot gain all these powers. Therefore, we only consider the threat when the adversary controls part of the powers of DirAuths. An adversary either has the ability to call functions of the SC that attempts to manipulate the network directory, or has the ability to eavesdrop on some relays in order to track the identity of clients.

Secondly, we consider that an adversary can control some of the trusted relays in the network. Unlike the Tor network, BADNET relays are deployed by DirAuths and have no incentive to disrupt the running of the network. However, there is a risk of IP exposure for relays as the exit relays directly communicate with the network service. After taking over a relay, an adversary can learn algorithms and keys for communications, upload incorrect data, forge malicious connections, and ultimately degrade the security and anonymity of the system.

Finally, we consider a censoring adversary. In BADNET, DirAuths are hidden behind the Ethereum system and directory services are provided by the SC for relay-and-client. The SC is a program running on Ethereum; once its creation is complete, the code cannot be modified and will run permanently with Ethereum. We assume that the SC follow the security attributes of the smart contract, but censors can monitor or reject traffic accessing Ethereum within a censored area. However, we assume that the censor is reluctant to block or significantly hinder the use of Ethereum. Otherwise, due to the large user base of cryptocurrencies, it may have negative consequences for some censored areas [7]. With regard to surveillance of Ethereum, we now formally define a Private Access (PA) Game.

Private Access (PA) Game. We say that the directory system in an anonymity network uses blockchain $\mathcal{B}$ to provide *private access* for the DirAuths, if given a pair of transactions TXN_{i_0}, TXN_{i_1} in $\mathcal{B}$, the adversary can not identify the transaction id choice of the DirAuths with probability significantly greater than that of a random guess (1/2). Formally, we want to demonstrate that any probabilistic polynomial time (PPT) adversary $\mathcal{A}$ has only a negligible advantage over random guessing on the following security game against a challenger $\mathcal{C}$ that accesses data stored on $\mathcal{B}$.

- $\mathcal{C}$ installs required software (i.e., DirAuth program). $\mathcal{A}$ stores the blockchain $\mathcal{B}$ to serve its content over the cryptocurrency P2P network, upon request.
- The adversary $\mathcal{A}$ may perform a polynomially bounded number of operations to select two indices i_0, i_1 of transactions in blockchain $\mathcal{B}$.
- At a desired time, $\mathcal{A}$ sends to $\mathcal{C}$ the indices i_0 and i_1, and $TXN_{i_0} = (i_0, d_0, u_0) \in \mathcal{B}$, $TXN_{i_1} = (i_1, d_1, u_1) \in \mathcal{B}$.
- $\mathcal{C}$ picks a bit $b \in_R \{0,1\}$ uniformly at random. $\mathcal{C}$ performs communication Γ to access transaction TXN_{i_b} from $\mathcal{A}$, then signals completion.
- $\mathcal{A}$ performs additional computations and outputs bit b' , his prediction of the bit b chosen by $\mathcal{C}$.

We define the advantage of $\mathcal{A}$ in the PA game to be:

$$\mathbf{Adv}^{PA}_{DirAuths}(\mathcal{A}) = |P(b' = b) - P(b' \neq b)|$$

We then introduce the following definition:

Definition 1. (Private Access) BADNET provides private access for the DirAuths if there exists no PPT adversary with a non-negligible advantage in the PA game.

3.3 System Goals

- *Unobservability of DirAuths.* An adversary cannot detect communication between DirAuths and other BADNET participants, even if it inspects the packets sent and received by the relay or client.
- *Relay protection.* Effective protection of the IP of relays while maintaining secure path selection to increase the difficulty for an adversary to monitor the relays.
- *Trustworthy bootstrapping.* An adversary is not able to manipulate the network directory, even in the presence of dishonest DirAuths.
- *Relay removal.* BADNET can remove bad relays via the SC without interrupting the running of the system.
- *Unblockability.* An adversary is unable or unwilling to block communications between clients and Ethereum, even if it can identify such communications.
- *Anonymity and unlinkability.* The anonymity and unlinkability of BADNET users should not be degraded.
- *Deployability.* BADNET can be deployed in a real-world scenario to provide anonymity services to users.

4 Design

We now describe the details of the BADNET design for anonymous communication. Our main idea is to create a decentralized medium on top of a public blockchain for curating network information in a trustworthy manner. We begin with preliminaries and initialization requirements. We then elaborate on a typical workflow of BADNET, involving four main participants: a set of DirAuths providing the trusted relay management service; the Ethereum system used to store network information; users who want to use the anonymity service; and relays who offer their bandwidth and CPU resources to tunnel traffic between users and destination websites.

4.1 Preliminaries

Broadcast encryption. The concept of broadcast encryption (BE) was firstly introduced in [21]. In BE schemes, a broadcaster encrypts messages and transmits them to a group of users who are listening to a broadcast channel and use their private keys to decrypt transmissions. At encryption time, the broadcaster can choose the set S of identities that will be able to decrypt messages.

We formally define the notion of broadcast encryption, following the scheme of Boneh, Gentry and Waters [8] and also adopting most of its notation. A BE scheme $\mathcal{BE}$ with maximal size n of the target set, consists of three algorithms $(\mathsf{Setup}, \mathsf{Encrypt}, \mathsf{Decrypt})$.

- **Setup**(n): Takes as input the total number of users n, and outputs a public key PK and n private keys $d_1, ..., d_n$.
- **Encrypt**(S, PK): Takes as input a subset $S \subseteq \{1, ..., n\}$ and a public key PK. It outputs a pair $(\mathtt{Hdr}, K)$ where $\mathtt{Hdr}$ is called the header and $K \in \mathcal{K}$ is a secret key for the symmetric encryption scheme.

  Let $\mathcal{E}_{sym}$ be a symmetric encryption scheme with keyspace $\mathcal{K}$, and algorithms SymEnc and SymDec. Let M be a message to be broadcast that should be decipherable precisely by users in S. Let $C_M \xleftarrow{R} \mathsf{SymEnc}(K, M)$ be the encryption of M under the symmetric key K. The broadcast to users in S consists of $(S, \mathtt{Hdr}, C_M)$.
- **Decrypt**$(S, i, d_i, \mathtt{Hdr}, PK)$: Takes as input a subset $S \subseteq \{1, ..., n\}$, a user identity $i \in \{1, ..., n\}$ and the corresponding private key d_i, a header $\mathtt{Hdr}$, and the public key PK. If $i \in S$, then the algorithm outputs a key $K \in \mathcal{K}$ which is then used to decrypt C_M to get the message M.

We always require $\mathcal{BE}$ to be perfectly correct, meaning that for all subsets $S \subseteq \{1, ..., n\}$ and all $i \in S$,

if $(PK, (d_1, ..., d_n)) \xleftarrow{R} \mathsf{Setup}(n)$ and $(\mathtt{Hdr}, K) \xleftarrow{R} \mathsf{Encrypt}(S, PK)$, then $\mathsf{Decrypt}(S, i, d_i, \mathtt{Hdr}, PK) = K$.

Access blockchain data. Blockchain is built on the idea of peer-to-peer networks, providing a shared and trusted ledger of transactions. There are two main types of nodes in a typical blockchain system, full nodes and light nodes. A full node stores all blockchain data, including block headers and transaction records, and the replicated ledger is broadcast to all full nodes., while a light node stores only block headers and relies heavily on requesting everything else from full nodes. But it enables users to participate in blockchain without the powerful hardware or high bandwidth. Therefore, the BADNET software application can connect to the Ethereum network by running Ethereum clients (e.g., Geth) in light mode and interact with the Ethereum client via the JSON-RPC API.

Unfortunately, few full nodes opt to serve light node data, which means that light nodes often fail to find peers. Although light node support is expected to develop rapidly in the near future, Ethereum does not currently support a large population of light nodes. Further, running a private Ethereum node can be complex and challenging for most BADNET users. We note that many node-as-a-service infrastructures such as Infura [25], QuikNode [48] run optimized nodes to serve users with instant access to the Ethereum blockchain. Most of them are used frequently, for example, Infura has served thousands of Ethereum users and handles billions of requests per day, as these services lower the barriers to blockchain-based applications and simplify access to on-chain data.

The main difference between writing and reading data in Ethereum is that writing data requires the creation of new transactions, thereby incurring fees, while reading data does not. Consequently, BADNET clients do not require any payment, but both relays and DirAuths have to write data to Ethereum and thus cost ETH. In this solution, we select one Ethereum testnet (e.g., Ropsten, Rinkeby, Kovan, etc.) to act as the infrastructure for BADNET, as testnets provide users with free cryptocurrency through faucets that can be used to pay for transaction fees. We note that there are many different types of faucets available online. We propose two strategies for relays and DirAuths to fetch cryptocurrency. First, the SC returns the fee for every successful transaction to the sender of this transaction. For another, relays and DirAuths periodically check the account balance and get cryptocurrency from a faucet when the balance is low. Note that these procedures are performed automatically. Although most faucets give a small amount of ETH for each request, it is sufficient to cover a relay's transactions for a considerable period of time.

Network information formatting. We note that Tor's relay selection strategy is based on some constraints for improving security and performance. For instance, the first hop on the path must be a guard relay, and the last hop must be an exit relay. Since the number of relays is significantly small compared to the potential number of users, Tor calculates relay weights that bias path selection so that relays with higher capacities are selected with a higher probability in order to enable overall load balancing. The client also can choose relays according to their geographical regions. To allow for complex path requirements, we organize network information into two separate databases: first, the so-called network directory database, and secondly, the network status database.

We call the network directory database a *Network Information with Multiple Entry* (NIME), which makes up of contact information representing all relays, such as their cryptographic identity keys and IP addresses. This database is dynamically updated as relays upload data periodically. We call each entry in the NIME a *Separable Relay Information* (SRI). Each SRI corresponds to a single relay, so the NIME comprises the complete set of all SRIs. We call the network status database a *Network Status Distribution* (NSD), which includes network status information. Note that the contents of the NSD may in different with the path-selection algorithm changes. For example, Tor's bandwidth-proportional

path selection algorithm requires the NSD to contain pre-relay bandwidths. However, in the case where clients completely choose nodes at random (for high anonymity), NSDs simply provide the total number of available relays.

NIMEs. Importantly, in BADNET, only relays need to download NIMEs. Relays are required to register at first bootstrap, and cannot fetch the entire NIME until they are approved and listed. Afterwards, relays fetch only the changes to the NIME once per epoch. BADNET does not allow entities outside the set of approved relays to access the NIME directly, since completely open relay information makes anonymity networks more susceptible to censorship.

SRIs. In contrast to Tor, we clarify the differences between an SRI and a relay entry that is the information about a single relay contained in the consensus document as follows.

First, each SRI includes an *index*, an integer value assigned when the relay successfully registers at the BADNET system. Once a relay changes its contact information, the relay will overwrite the expired SRI with the updated data directly based on the index. As we shall see in Section 4.5, these index enable clients to indicate which relay to entend their circuit to without maintaining the NIME locally.

Second, unlike complex Tor server descriptors, each SRI contains only the necessary contact information about a BADNET relay (e.g., IP address, ports, public keys) and a counter value indicating the version of S at the time an SRI was uploaded. BADNET relays update SRIs when their contact information changes or when the relay group is updated.

Third, each SRI is stored in the form of encrypted format. Before writing data to the public blockchain, relays always execute a hybrid encryption, i.e., broadcast encryption creating a symmetric key and a cipher-key-based encryption over information.

NSDs. In BADNET, only clients are required to download NSDs. Clients fetch the NSD once per epoch to be aware of the up-to-date network status. In addition to advertising the relay status for path selection, the NSD includes SRI digests (i.e., the cryptographic hash of SRIs) produced by the relays themselves. These SRI digests allow clients to verify SRIs without downloading the full NIME, and will be updated as relays update their SRIs.

Counters. The SC also maintains a counter field that represents the version number of the relay index set S. More concretely, the initial value of the counters is 0; afterwards, once the relay group is updated, i.e., new relays are added or bad relays are removed, the counters' value will automatically increment by one. The version control of S is to ensure the consistency of S during the encryption and decryption process in the BE system.

By manipulating the directory, an adversary is able to admit or direct more traffic to malicious Tor relays. However, in the BADNET design, both NIMEs and NSDs are stored in the blockchain ledger (as depicted in Figure 2), ensuring

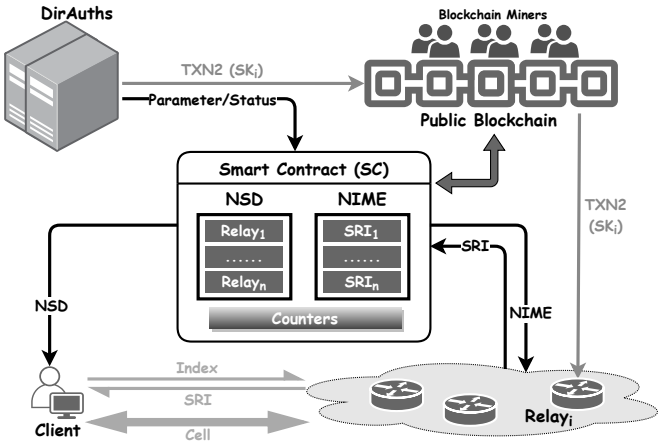


Figure 2: BADNET data exchange workflow: (1) private keys used for the BE system is transmitted to relays by DirAuth using blockchain transactions; (2) relays upload their SRIs and download NIMEs; (3) clients fetch NSDs from the SC and request SRIs from relays according to indexes.

stable and efficient downloading with the benefits of tamper resistance and avoiding single or few point-of-failure vulnerabilities. Additionally, to prevent malicious relays from attacking clients by giving them SRIs nobody else uses, clients should compare the hash of each received SRI with its SRI digest to guarantee data integrity. In this case, the number of SRI digests in the NSD is equal to N_R, where N_R denotes the number of available relays in the BADNET system.

4.2 Initialization

Creation of smart contracts. BADNET is designed with the assumption that a directory system exists and it gives each client the same view of the network while preventing adversaries from using it to track users. The directory system is essentially a smart contract running on a decentralized blockchain that is used to curate all available relays. As the first step for running a BADNET anonymity network, the DirAuth creates a smart contract (SC) on the public blockchain such as Ethereum and Solana. Afterwards, all users of said blockchain can access the SC from any blockchain node around the world. Operations on NIMEs and NSDs are performed by calling *smart contract functions*, i.e., bootstrapping the anonymity service from a decentralized system operated by multiple unrelated entities, rather than directly from any fix-IP-address DirAuth.

Building a broadcast encryption system. A BE system generally involves an authority: the Private Key Generator (PKG). The PKG grants new members capability of decrypting messages by offering each new member (with identity i) a decryption key d_i. We propose to make the BADNET DirAuth act as the PKG of the BE system. In the setup phase,

Algorithm 1: Relay Registration Procedure

1 **Function** `relay_register`$([R, SC, DA])$**:**
2 $(R.ECC_{PK}, R.ECC_{SK}) = R.\text{ecc_key_gen}()$;
3 $R.ACCT_{Addr} \leftarrow R.ECC_{PK}$;
4 $DA_{PK} = R.\text{get_dirauth_pk}()$;
5 $MSG \leftarrow R.\text{PKEnc}(DA_{PK}, R.CRDL) + R.ECC_{PK}$;
6 $[MSG]_{sig(R.ECC_{SK})} \rightarrow SC$;
7 $\varepsilon \leftarrow DA.\text{PKDec}(DA_{SK}, \text{extract_data}(SC.TXN_1))$;
8 **if** $DA.\text{verify_credential}(\varepsilon)$ **then**
9 $[DA.\text{success}(R.ACCT_{Addr})]_{sig(DA.ECC_{SK})} \rightarrow SC$;
10 $[DA.\text{PKEnc}(R.ECC_{PK}, BE_{SK})]_{sig(DA.ECC_{SK})} \rightarrow SC$;
11 $R.\text{PKDec}(R.ECC_{SK}, \text{extract_data}(SC.TXN_2))$;
12 **else**
13 $ABORT$;
14 **end**

the DirAuth sets $n = 10,000$ to generate a public key and 10,000 private keys. The public key, with a size of roughly 92KB, will be shipped with the BADNET software, so that all relays use the same pre-distributed PK in algorithms Encrypt and Decrypt. For the cases $n = 50,000$ and $n = 100,000$, the size of the public key is approximately 205KB and 290KB, respectively. The new relay R_j joins the BE system by obtaining the private key d_j transmitted by the DirAuth from the blockchain channel. The size of one private key is always about 311B.

4.3 Relay Registration and Advertisement

Relay registration. Unlike Tor, BADNET relays are required to be approved by the DirAuth before contributing idle bandwidth and CPU resources to the network. Algorithm 1 presents the pseudocode of the relay registration. A relay first generates a blockchain account with corresponding public/private key pair (Lines 2-3). It then sends a registration transaction TXN_1 signed by its blockchain private key to the SC, including its credential (CRDL), which is used by the DirAUth to verify and certify a new relay. The data of TXN_1 is encrypted by a public key encryption (PKE) algorithm (Line 6). There are many means for relays to obtain credentials, e.g., off-chain payment channels [16, 47]. Many anonymity services such as JonDonym, the system is run and maintained by a company, but the nodes are operated by other independent entities. Therefore, the credentials can be provided by the company to the organization of relays. These organizations must abide by a strict set of rules that governs how to operate the relays, and prohibits them from saving connection data or exchanging such data with others. To defend against malicious relay operators who have credentials acquired by illegal means, the DirAuth must be able to remove them without disrupting system operations, as we shall discuss in 4.6.

For a successful verification (Line 8), a new relay record is created on the NIME as well as the NSD, respectively, and is linked with the relay's identifier. A relay is uniquely identified in BADNET by its blockchain address; this address is recorded into the SC and will be used as the relay's permanent identifier (Line 9). Then, the relay will be assigned a previously unused index $i \in 1, ..., n$ and given the corresponding secret key d_i, while the broadcast set S is updated by the SC to include i. All members of the current relay group can discover this change in S by accessing the SC. The d_i is transmitted via the transaction TXN_2 and is also encrypted by PKE. Note that all PKE algorithms in our current implementation use *Elliptic Curve Cryptography* (ECC), which offers the same level of security using a small size key as other PKE algorithms with large size keys.

Relay advertisement. Once a relay has registered in the BADNET network, the relay is added to the BE system and further advertises its information to other approved relays. Algorithm 2 presents the pseudocode of this process. More concretely, the relay first accesses the SC to obtain the up-to-date index set S (Line 3). Then, it sends an advertisement transaction, containing the relay information (i.e., its encrypted SRI and status) and SRI digest, to the SC (Line 6). All relays in BADNET will then obtain this SRI upon decryption (Line 12). Note that SRI_R is stored in the NIME in the form of encryption, while the relay status (i.e., ST_R) and the SRI digest (i.e., $\text{hash}(SRI_R)$) are directly stored in plaintext in the NSD. The ST_R is used for path selection, while the $\text{hash}(SRI_R)$ is used to verify SRI_R during circuit construction. Afterwards, if the SRI changes, the relay immediately advertises a new SRI to replace the old one by sending a transaction to the SC. Additionally, in each epoch (like 18 hours in Tor), relays check if the S has changed, and if so, re-upload the SRIs, otherwise, simply report their latest status such as bandwidth.

Algorithm 2: Relay Advertisement Procedure

1 **Function** `relay_advertisement`$([R, SC])$**:**
2 $BE_{PK} = R.\text{get_broadcast_encryption_pk}()$;
3 $S = R.\text{get_broadcast_encryption_group}(SC)$;
4 $(K, HDR) = R.\text{Encrypt}(S, BE_{PK})$;
5 $MSG \leftarrow R.\text{SymEnc}(K, SRI_R) + ST_R$;
6 $[MSG, HDR, \text{hash}(SRI_R)]_{sig(R.ECC_{SK})} \rightarrow SC$;
7 **Function** `relay_download`$([R_x, SC])$**:**
8 $(C_{SRI}, HDR) \leftarrow R_x.\text{extract_data}(SC)$;
9 $BE_{PK} = R_x.\text{get_broadcast_encryption_pk}()$;
10 $S = R_x.\text{get_broadcast_encryption_group}(SC)$;
11 $K \leftarrow R_x.\text{Decrypt}(S, i, d_i, HDR, BE_{PK})$;
12 $SRI_R \leftarrow R_x.\text{SymDec}(K, C_{SRI})$;

4.4 Client Request

Downloading NSD. Before routing traffic using BADNET relays, clients must be fully aware of the global network status to prepare for path selection. For scalability reasons, BADNET does not provide clients with the NIME; instead, it introduces a status view, the NSD, to discover forwarder nodes. NSDs cover the characteristics of the relays required by the path selection algorithm. The advantage of this design is that the DirAuth is able to remove bad (e.g., malicious, expired, offline) relays, but does not have the capability to influence the path, i.e., the path selection process is completely client-driven and allows for complex path requirements.

Like Tor, the BADNET client also selects the path from all available relays, remaining secure against epistemic attacks. Specifically, the client first chooses a set of (typically three) guard relays as the first hop of the path in a given period. Then, the client chooses for the last hop a relay that allows connections to the destination port and IP. For the middle position, any relay might be selected. As we discussed in Section 4.1, every SRI contains an index, corresponding to a single relay. Clients fetch relay statuses from the latest NSD and select relays based on the purpose of communications. Regardless of the complexity of the client's requirements, a path can be simply represented by a sequence of indexes.

Bootstrapping the connection to guards. When a client is startup for the first time, it does not have any information locally that can be used to establish a connection to the guard relay. Bootstrapping the first connection is an open problem for the anonymity network, so we propose several options for clients to bootstrap. But note that such bootstrapping is comparatively infrequent, as clients maintain long-lived guard relays to prevent enumeration attacks.

(i) Building from trusted guards. Many anonymity networks hard-code a list of stable relays in the client software as a bootstrapping mechanism. To bootstrap a first connection, BADNET clients can build a circuit using one of these pre-configured relays as the first hop. Upon completion of the circuit, clients can extend it to an additional relay that is suitable to serve as the first hop for future circuits. Then, the client can remember this relay's SRI, throw away the circuit, and build new circuits with this relay as their guard. In addition, BADNET can publish a list of relays on the blockchain. This list will be updated periodically by the DirAuth to mitigate an adversary efforts to discover user identities by monitoring relays.

(ii) Requesting designated guards via blockchain. For a set of privacy-conscious users who would not like to direct their traffic to public relays, BADNET also supports client bootstrapping using blockchain as a non-interactive transmission channel. For instance, the client creates a one-time blockchain account, and then employs it to send a *request transaction* to the SC containing the index value of selected guard relays. Shortly afterwards, the DirAuth will return their SRIs by sending a *response transaction*. Finally, the client can build circuits based on these SRIs.

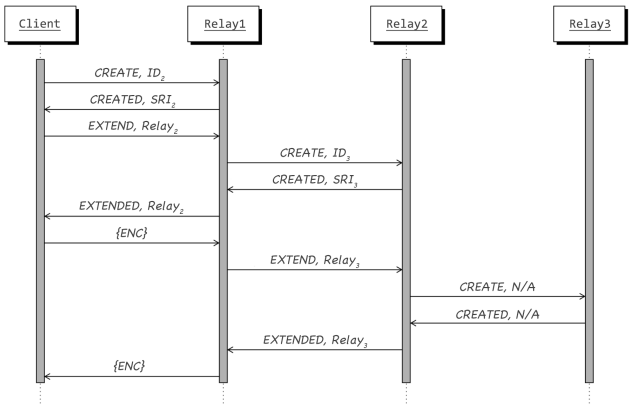


Figure 3: The client builds a three-hop circuit.

4.5 Circuit Construction

Like Tor, BADNET circuits are constructed incrementally: the client negotiates a symmetric key with every relay in the chosen path, one hop at a time, and uses each partially created circuit to communicate with the next hop. To begin creating a new circuit, a Tor client sends a *CREATE* cell, containing the first half of the DH handshake (g^x), to the first node, and then receives a *CREATED* cell, containing the response handshake (g^y). Importantly, the format of *CREATE* and *CREATED* is extensible by design, so we add a field to each of them indicating the next-hop index and the next-hop SRI, respectively.

Figure 3 presents the workflow of BADNET's circuit construction. As we described in Section 4.4, Client has sufficient information to establish a connection to Relay_1. In preparation for extending the circuit to Relay_2 (with the index ID_2), Client sends a *CREATE* cell including ID_2 and g^x to Relay_1. To find the relay chosen by Client, Relay_1 looks up ID_2 in the NIME (downloaded from the SC) for the unique SRI_2 that corresponds to the next hop Relay_2. Relay_1 then responds with a *CREATED* cell with SRI_2 and g^y, and Client uses the digest for SRI_2 in the NSD to verify the correctness of received SRI_2 (see Section 4.1). Finally, Client extends the circuit to Relay_2 using SRI_2. Similarly, Client fetches SRI_3 from Relay_2 and extends the circuit to Relay_3. But Client does not request SRI from Relay_3 for the next hop, since it is the last hop. After successfully building circuits, Client forwards onion packets by the constructed circuit to access the censored services.

4.6 Group Update

In many anonymity networks such as Tor, there is no formal procedure to apply to be a relay operator. It is trivial for adversaries to control some relays to perform end-to-end

correlation attacks [30, 44]. To mitigate this problem, Tor has some manual oversight of the network. Bad actors are removed when malicious behavior is spotted, which is a constant task. Unfortunately, removed relays is still able to learn about the network information, which is not applicable to the BADNET's design goals. We thus use a BE system to prevent removed relays from obtaining relay information to avoid continuous monitoring and blocking of anonymity services.

The goal of the BE system is to allow BADNET to remove relays (e.g., malicious relays, expired relays, offline relays, etc.) at minimal cost, and without disrupting the operation of the system. To achieve this goal, the SC maintains a health flag ($\mathtt{R}_{\mathtt{health}}$) for each registered relay with an initial value of *true*. The intuition is that relays which are assessed as requiring removal would no longer be able to decrypt to obtain SRIs. Therefore, by modifying $\mathtt{R}_{\mathtt{health}}$ to $false$, a relay R_j with index $j \in \{1,...,n\}$ is removed from the relay group and j is removed from the set S. Afterwards, as the relays are periodically updated (e.g., 18-hour period), R_j cannot decrypt any SRI.

When DirAuths finds a bad relay R_j with index j by active detection such as reachability test, bandwidth measurement [58]:
[1] Change R_j's $\mathtt{R}_{\mathtt{health}}$ to $false$, and counters thus plus one.

When relay R_k updates SRI periodically:
[2] R_k generates the latest relay index set $\mathtt{S}_{\mathtt{new}}$ based on the $\mathtt{R}_{\mathtt{health}}$ data of relays and gets the value of counters $cntr$.
[3] R_k uses $\mathtt{S}_{\mathtt{new}}$ as an input to the encryption algorithm for the BE system to generate a new symmetric key k.
[4] R_k uploads the up-to-data SRI (including $cntr$) to the SC after encrypting it with k.

When relay R_l downloads NIME from the SC periodically:
[5] R_l generates the latest relay index set $\mathtt{S}_{\mathtt{new}}$ based on the $\mathtt{R}_{\mathtt{health}}$ data of relays and gets the value of counters $cntr_{new}$.
[6] For each SRI in the NIME, R_l extracts its counter value $cntr_{old}$.

If $cntr_{old} = cntr_{new}$, then $\mathsf{Decrypt}(S_{new}, l, d_l, \mathtt{Hdr}_1, PK) = K_1$, otherwise, using $cntr_{old}$ to roll back $\mathtt{S}_{\mathtt{new}}$ to $\mathtt{S}_{\mathtt{old}}$, and then $\mathsf{Decrypt}(S_{old}, l, d_l, \mathtt{Hdr}_1, PK) = K_1$.

When relay R_j try to download NIME from the SC:
[7] R_j cannot decrypt the encrypted data uploaded by R_k.

If $j \notin S_{cur}$ and $(\mathtt{Hdr}_2, K_2) \xleftarrow{R} \mathsf{Encrypt}(S_{cur}, PK)$, then $\mathsf{Decrypt}(S_{cur}, j, d_j, \mathtt{Hdr}_2, PK) \neq K_2$.
[8] Once all relays have completed SRI updates, R_j will not be able to learn any information from the NIME.

5 Security Analysis

To demonstrate the benefit of BADNET, this section analyzes several existing security concerns in Tor and provide a security analysis for BADNET. We further focus on attacks that might be caused by the differences between BADNET and Tor, and discuss briefly how BADNET defends against them. Finally, we look at how hiding DirAuths can help to protect against tracking to the BADNET client.

Single or few point-of-failure vulnerabilities. Tor runs redundant (but only nine) authorities that vote on the state of the network to enhance the reliability of the directory system, but each of these servers is a single point-of-failure vulnerability. BADNET is built on a public blockchain distributed across thousands of peer nodes around the world. Each node has the same copy of the data. With the decentralized consensus mechanism of blockchain, the directory system in BADNET is resilient to up to n failures when a blockchain contains $2n+1$ nodes, where $n \in N_+$.

Denial-of-service attacks. As a result of its completely transparent network information, DoS attacks against Tor are viable and less complex [5, 9, 27, 28]. BADNET mitigates DoS attacks in anonymity networks from two aspects.

BADNET is a relatively closed system as the barrier to becoming a relay operator is heightened. We employ various social engineering techniques such as financial rewards to attract a large number of honest relay operators. These operators are unrelated entities or private individuals whose identities are credentialed and overseen by the community of trusted operators. Further, the information of BADNET relays is not publicly available. For an external adversary that can use a distribute network of machines capable of sending traffic into the network [28], it cannot decrypt the encrypted content published on blockchain to acquire the NIME due to without knowledge of the decryption keys. A malicious client can exhaust the relay's memory by exploiting end-to-end reliable data transport [27]. However, BADNET clients cannot fetch the NIME directly, and can only learn more relays by constantly building new circuits, increasing the difficulty for adversaries to disable relays. Unfortunately, a malicious relay can easily fetch the list of relays, which provides the same level of security as Tor.

On the other hand, the directory data in BADNET is stored on a public blockchain. Traditional DoS attacks [17] are difficult to scale to decentralized blockchain systems with a large number of nodes. However, blockchains are not completely immune to DoS attacks. There is a body of research on blockchain DoS security, but these known DoS attacks that seek to stop cryptocurrency blockchains are prohibitively costly. For example, BDoS [40] is a mining-based attack that aims to incentivize rational miners to stop mining. An adversary could launch a BDoS on Bitcoin (in Mar. 2020) at a cost of nearly $1M per day, which is unrealistic in practice. Other attacks [12, 13] attempt to deny blockchain transaction handling by saturating the limited transaction throughput. However, the adversary simply raises the cost of writing to the blockchain. Clients can still read data from the blockchain. Additionally, BADNET completely masks the identity of DirAuths and participants will not establish connections with them. This level of concealment ensures that an adversary is not able to DoS attack against the DirAuths.

Directory manipulation attacks. An adversary can manipulate the directory by taking over DirAuths. Tor uses a multi-party voting mechanism to diffuse trust in a single DirAuth. During the voting process, each DirAuth collects the signed vote documents of other DirAuths and then generates the same consensus based on a well-specified voting algorithm. But subverting a DirAuth can partially influence the final directory, and subverting a majority of DirAuths (e.g., five) can admit as many malicious relays in the directory as possible [18]. As we described above, the concealment of DirAuths keep them from becoming a vulnerable BADNET component. By placing directory data into a public blockchain, BADNET not only prevents adversaries from modifying the directory, but also enables a transparent directory system where all code and operations can be tracked and verified publicly.

Epistemic attacks. As pointed out in the past [14, 15], clients' partial knowledge of the relays belonging to the anonymity network enables epistemic attacks. Therefore, every client has to be selecting relays from the same probability distribution. Tor employs a centralized directory infrastructure that allows each client to know about each relay in order to defend against epistemic attacks. Similarly, BADNET is not vulnerable to such attacks because there is no difference in the set of relays known to each client; downloading the same NSD makes the adversary's view of distribution to uniform, and the path is selected by the client. As the NSD contains the status of all relay, the client can select any SRI in the NIME.

Route capture attacks. A client's chosen path can be replaced or influenced by a malicious intermediary [61]. To defend against route capture attacks, Tor clients need to check consensus signatures after downloading the latest consensus document. Although BADNET clients do not fetch the NIME from the SC, they can validate that their path has not been influenced by an intermediary. As previously established, the client uses SRI digest in the NSD to validate that their choice of ID corresponds to the SRI_x returned by the relay. Therefore, a malicious relay or intermediary cannot return only the SRI of malicious relays as the result.

Unlinkability of directory requests. Unlinkability requires that an adversary should not be able to distinguish whether two or more communicating entities are related [46]. In the context of BADNET, our goal is to prevent any party (internal or external to the network) from determining the identity of clients. Public anonymity networks like Tor do not hide the fact that clients are using the anonymity service. Rather, their anonymity relies on concealing individuals from the large number of clients. It works for the Tor network with million-level anonymity sets, but is dangerous for relatively private networks. More concretely, by monitoring the directory system, an adversary can learn the membership of the set of all clients in the network. Even for a private anonymity network that does not publish its relays and DirAuths, an adversary (e.g., an eavesdropper on websites) is able to discover the identity of DirAuths by tracing an exit relay. Therefore, we aim to provide unlinkability between BADNET participants and the DirAuths.

We now proof that BADNET provides private access for the DirAuths to guarantee their unlinkability to relay-and-client. Following the private access game (Definition 1), the DirAuths first download the entire blockchain before accessing all data-containing transaction TXN_i. Thus, the only possible communication Γ is the entire log of transactions in blockchain $\mathcal{B}$ until all TXN_i have been observed. Therefore, $P(\Gamma = \mathcal{B} \mid b) = P(\Gamma = \mathcal{B}) = 1$ for all of $\mathcal{C}$'s choices of b, i.e., no matter the choice of b, $\mathcal{A}$ always observes the same communication. This implies that knowledge of Γ does not provide $\mathcal{A}$ with an advantage in the choice of b. Formally, for any PPT $\mathcal{A}$, the conditional probability of guessing $\mathcal{C}$'s b value given communication Γ is, according to Bayes' theorem, $P(b' = b \mid \Gamma = \mathcal{B}) = P(\Gamma = \mathcal{B} \mid b)/P(\Gamma = \mathcal{B})$. Since the choice of b is random (see the PA game in Section 3.2), $P(b) = 1/2$, thus $P(b' = b \mid \Gamma = \mathcal{B}) = 1 \times (1/2)/1 = 1/2$. Thus, we have that for any adversary $\mathcal{A}$:

$$\begin{aligned}\mathbf{Adv}^{PA}_{DirAuths}(\mathcal{A}) &= |P(b' = b) - P(b' \neq b)| \\ &= |P(b' = b) - (1 - P(b' = b))| \\ &= |2 \times P(b' = b \mid \Gamma = \mathcal{B}) - 1| = 0\end{aligned}$$

6 Evaluation

On the basis of two open source projects, Ethereum and Tor, we create a realistic evaluation environment for BADNET by building a prototype system in C, Python and Solidity. Our system consists of 20 clients, 32 relays, and one SC developed on Ropsten. Users used the network for web browsing, file downloading, messaging, etc. At the start of our experiments, the system worked stably for two months; afterwards, it run until September 2022, with a total of 9,204 transactions. Since the Ropsten testnet has been discontinued in October 2022, we moving to the Rinkeby Ethereum testnet.

6.1 Circuit Construction Performance

We now compare the performance of BADNET to Tor in terms of circuit construction time. We evaluate latency through high-fidelity network simulation using Shadow [26], a network simulator that runs Tor code in a private network.

We conducted group experiments according to the type of network, with a total of two groups (i.e., Tor and BADNET). Leveraging Shadow, we configure a private Tor network and a private BADNET network, respectively. They both consist of 6 relays and 6 clients. Ten experiments were performed in every group. Each experiment enables every client to build ten circuits, so a total of 60 circuits were built by each group per experiment. Overall, the average time to build one circuit in Tor is approximately 603 milliseconds, while the average time for BADNET is about 1403 milliseconds. However, the

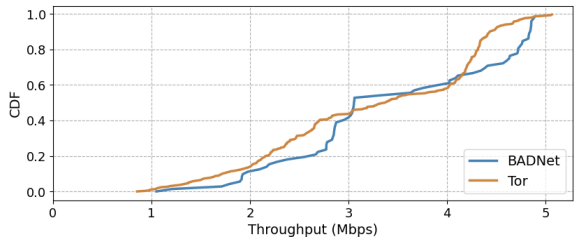


Figure 4: CDF of client throughput

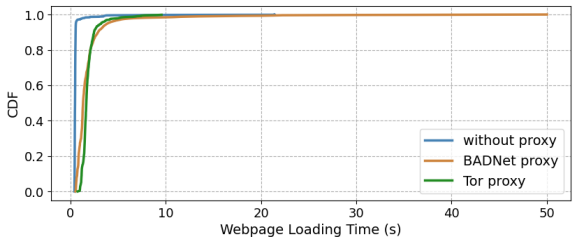


Figure 5: CDF of page loading time

delays incurred by circuit construction do not degrade the end-to-end performance of BADNET, and clients typically pre-build multiple circuits in preparation for communication.

6.2 End-to-end Performance

To quantify end-to-end performance in a wide-area network, the relays and clients in our BADNET prototype are run by Alibaba Cloud ECS instances located worldwide, such as China, Australia, Germany, etc. At the client side, we use a Tor Browser and BADNET to download a 10 MB file from an HTTPS server, respectively. For each measurement, we establish a new circuit. Figure 4 shows the CDF of throughput for BADNET and Tor. BADNET exhibits 3.7% higher throughput (3.05Mbps) than Tor. From the results we infer that the communication performance of BADNET is not worse than Tor. We also evaluate the web latency when clients connect to a website through BADNET. We perform 1,000 measurements for each client that access the website without a proxy, with a Tor proxy, and with a BADNET proxy. Figure 5 shows the distribution of the web page loading time for some popular websites. We find that 90% of non-proxy connections have a latency of under 0.54s. But 90% of BADNet connections have a latency of less than 3s, which is comparable to Tor. Clearly, using proxies to improve anonymity increases latency, but it can still be within an acceptable level.

6.3 Directory Service Performance

While the limitations of small-scale anonymity networks can be solved using blockchain technology, blockchains also have limitations. Block time, the time required to generate the next block of transactions in the chain, would introduce additional waiting time, and storing data in blockchain would require payment of transaction fees. We now evaluate the performance of the directory system in terms of latency, bandwidth, and financial overhead.

6.3.1 Latency Evaluation

We now evaluate the latency incurred by relays and clients of BADNET when interacting with blockchain. We assess the latency experienced by relays uploading the latest SRI and the impact of these transactions on the Ropsten Testnet, and the latency experienced by clients in directory services. Due to most Testnets only select a small number of nodes to validate transactions and create blocks, the transaction frequency tends to be lower than that of the Ethereum Mainnet [64].

Relays. The number of blockchain transactions introduced by BADNET's directory service increases as the number of relays increases. To quantify the impact of these transactions on Ethereum, we use a few relays to simulate different numbers of relays by shortening the relay's upload interval.

Generally, if the contact information has not changed, the BADNET relay simply reports the latest status, which is much less data than an SRI. But in the worst case, every relay needs to update the SRI at each epoch. Firstly, we select 13 relays that upload SRIs every 8 minutes, so they would upload SRIs a total of 1,755 times in an 18-hour period, which is close to a quarter of Tor's scale. Secondly, we adjust the upload period to 4 minutes, so these routers would upload 3,510 times in 18 hours, nearly half the scale of Tor. Finally, we expand to 32 relays and set the upload interval to 5 minutes to simulate the current scale of Tor by uploading 6,912 SRIs. Note that blockchain transactions will fail in rare cases due to rejection. In our experiments, these failures were not handled due to the small upload interval, but in practice the relay will resend a transaction after detecting a failure to ensure the data uploaded successfully.

For the three transaction scales, Figure 6 shows the CDF of transaction latency ($T_{latency}$). We measure the time from the creation of a transaction until the return of the transaction receipt. For each transaction, we set the *timeout* parameter for waiting for a transaction receipt to 5 minutes. Most transactions (actually 82.8%) were committed within this time. We see that the median of $T_{latency}$ increases as the number of transactions doubles, which indicates that the growing fraction of transactions that require a longer wait for a confirmation. The bottleneck in Ethereum's transaction processing capacity [19] results in a large number of transactions that cannot be fit into the block rapidly, which could be a hindrance to the scalability of BADNET. However, the latency of these transactions does not degrade client performance because clients do not need to send any transaction to the SC.

Clients. To quantify the latency overhead of clients, we test the effect of different numbers of relays on client performance

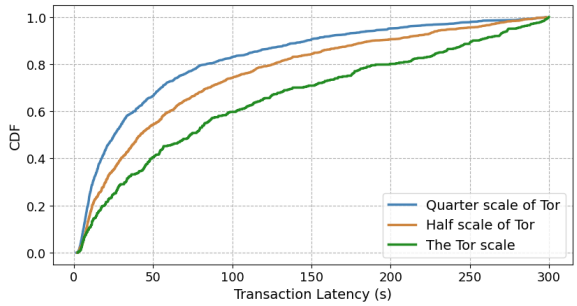


Figure 6: Latency overhead for committing transactions under different network scales of BADNET

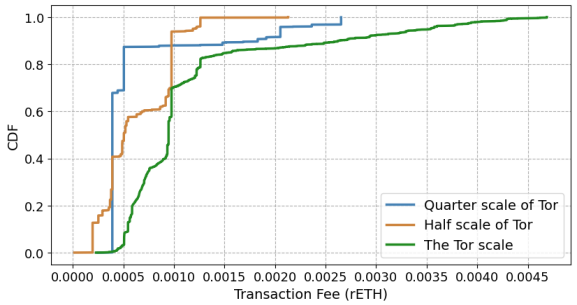


Figure 7: Cost for committing transactions under different network scales of BADNET

by running test contracts corresponding to 1,750, 3,500, and 7,000 registered relays, respectively. We find that the latency of downloading NSD is independent of the number of relays, and the average time elapsed for downloading NSD is about 0.29s. The results indicate that BADNET's directory system is practical in practice.

6.3.2 Financial Evaluation

BADNET should require clients as few configuration decisions as possible in order to increase usability. Therefore, clients typically do not send transactions and do not pay fees. We thus evaluate the financial cost for the the process of relay registration and advertisement.

Relay registration. Relay registration is a non-interactive communication process between the relays and DirAuths. More concretely, a relay sends a request transaction TXN_1, and then the DirAuths respond with a transaction TXN_2 (as described in Section 4.3). We conduct one hundred experiments for both types of transactions. Overall, the experimental results show that the average cost for sending a TXN_1 is 0.000023 ETH, while the average cost for a TXN_2 is 0.000415 ETH. For a relay, sending a TXN_1, i.e., registration, is a one time operation that will not dominate its economic cost.

Relay advertisement. As described in 6.3.1, we instantiate three kinds of transaction scales with several relays. For each transaction, the *gasPrice* parameter is set to the current average gas price. Figure 7 shows the CDF of transaction fee for uploading SRI (T_{cost}). The results indicate that the gas price increases with the network throughput, which is obviously due to Ethereum's Proof-of-Work (PoW) consensus mechanism. For BADNET equivalent to Tor scale, the average fee of transactions is 0.001178 ETH.

We let BADNET to use a Ethereum Testnet such as Ropsten, Rinkeby to implement the directory system. Testnet coins are separate and distinct from the official (Mainnet) coins, do not have value, and can be obtained freely from *faucets*. It fundamentally removes the economic cost for running a BADNET system. More importantly, there are many Testnet faucets available publicly, and a large number of users request Testnet coins from them per day. Therefore, even if an adversary tracks the blockchain address of a relay, the adversary cannot link the address to the real identity of the relay operator. Note that blockchain Testnets do have fewer nodes than Mainnets, but they are also more decentralized than 9 dedicated servers fixed in the Tor directory system.

7 Conclusion

This paper proposes a new mechanism to implement relay discovery for anonymity networks on top of the blockchain, which utilizes the basic idea of Tor, but hides the central directory authority behind a decentralized Ethereum system while stripping all connections between authority and relay-and-client. By taking advantage of Ethereum smart contracts to provide directory services, BADNET increases the censorship resistance of a Tor protocol-based anonymity network. Additionally, BADNET protects the confidentiality of relay data with broadcast encryption and a new circuit building protocol. We implemented a preliminary system prototype of BADNET based on Tor and Ropsten, demonstrating that our solution is practical in real-world scenarios.

References

[1] I2P. https://geti2p.net/, 2022.

[2] Namecoin. https://www.namecoin.org/, 2022.

[3] Carlos Aguilar-Melchor and Philippe Gaborit. A Lattice-Based Computationally-Efficient Private Information Retrieval Protocol. In *WEWoRC*, 2007.

[4] Greubel Andre, Dmitrienko Alexandra, and Kounev Samuel. SmarTor: Smarter Tor with Smart Contracts: Improving resilience of topology distribution in the Tor network. In *Annual Computer Security Applications Conference*, pages 677–691, 2018.

[5] Marco Valerio Barbera, Vasileios P Kemerlis, Vasilis Pappas, and Angelos D Keromytis. Cellflood: Attacking

tor onion routers on the cheap. In *European Symposium on Research in Computer Security*, pages 664–681, 2013.

[6] Kevin Bauer, Damon McCoy, Dirk Grunwald, and Tadayoshi Kohno andcDouglas Sicker. Low-Resource Routing Attacks Against Tor. In *ACM workshop on Privacy in electronic society*, pages 11–20, 2007.

[7] BI India Bureau. No crypto ban in India — SEBI will reportedly be pulled in to regulate the sector. https://www.businessinsider.in/cryptocurrency/news/no-crypto-ban-in-india-sebi-will-reportedly\-be-pulled-in-to-regulate-the-sector/articleshow/88065166.cms, 2021.

[8] Dan Boneh, Craig Gentry, and Brent Waters. Collusion Resistant Broadcast Encryption with Short Ciphertexts and Private Keys. In *Annual international cryptology conference*, pages 258–275, 2005.

[9] Nikita Borisov, George Danezis, Prateek Mittal, and Parisa Tabriz. Denial of service or denial of security? In *Proceedings of the 14th ACM conference on Computer and communications security*, pages 92–102, 2007.

[10] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. FORESHADOW: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution. In *USENIX Security Symposium*, pages 991–1008, 2018.

[11] David L Chaum. Untraceable electronic mail, return addresses, and digital pseudonyms. *Communications of the ACM*, 24(2):84–90, 1981.

[12] ConsenSys Diligence. DoS with Block Gas Limit. https://consensys.github.io/smart-contract-best-practices/attacks/denial-of-service/#dos-with-block-gas-limit, 2018.

[13] ConsenSys Diligence. DoS with (Unexpected) revert. https://consensys.github.io/smart-contract-best-practices/attacks/denial-of-service/#dos-with-unexpected-revert, 2018.

[14] George Danezis and Richard Clayton. Route fingerprinting in anonymous communications. In *Sixth IEEE International Conference on Peer-to-Peer Computing (P2P'06)*, pages 69–72. IEEE, 2006.

[15] George Danezis and Paul Syverson. Bridging and fingerprinting: Epistemic attacks on route selection. In *International Symposium on Privacy Enhancing Technologies Symposium*, pages 151–166. Springer, 2008.

[16] Christian Decker and Roger Wattenhofer. A fast and scalable payment network with bitcoin duplex micropayment channels. In *Symposium on Self-Stabilizing Systems*, pages 3–18. Springer, 2015.

[17] Department of Homeland Security. Understanding Denial-of-Service Attacks. https://www.cisa.gov/uscert/ncas/tips/ST04-015, 2018.

[18] Roger Dingledine, Nick Mathewson, and Paul Syverson. Tor: The second-generation onion router. Technical report, Naval Research Lab Washington DC, 2004.

[19] Tien Tuan Anh Dinh, Ji Wang, Gang Chen, Rui Liu, Beng Chin Ooi, and Kian-Lee Tan. Blockbench: A framework for analyzing private blockchains. In *ACM International Conference on Management of Data*, pages 1085–1100, 2017.

[20] Nathan S. Evans, Roger Dingledine, and Christian Grothoff. A Practical Congestion Attack on Tor Using Long Paths. In *USENIX Security Symposium*, pages 33–50, 2009.

[21] Amos Fiat and Moni Naor. Broadcast Encryption. In *Annual International Cryptology Conference*, pages 480–491, 1993.

[22] Michael J Freedman and Robert Morris. Tarzan: A peer-to-peer anonymizing network layer. In *Proceedings of the 9th ACM conference on Computer and communications security*, pages 193–206, 2002.

[23] Ian Goldberg. Improving the Robustness of Private Information Retrieval. In *IEEE Symposium on Security and Privacy*, pages 131–148, 2007.

[24] Siobhan Gorman and Jennifer Valentino-DeVries. New details show broader nsa surveillance reach. *The Wall Street Journal*, 20, 2013.

[25] Infura. Ethereum api. https://infura.io/product/ethereum, 2022.

[26] Rob Jansen and Nicholas Hopper. Shadow: Running Tor in a Box for Accurate and Efficient Experimentation. In *Network and Distributed System Security Symposium*, 2012.

[27] Rob Jansen, Florian Tschorsch, Aaron Johnson, and Bjorn Scheuermann. The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network. In *Network and Distributed System Security Symposium*, 2014.

[28] Rob Jansen, Tavish Vaidya, and Micah Sherr. Point Break: A Study of Bandwidth Denial-of-Service Attacks against Tor. In *USENIX Security Symposium*, pages 1832–1840, 2019.

[29] Aaron Johnson, Rob Jansen, Nicholas Hopper, Aaron Segal, and Paul Syverson. PeerFlow: Secure Load Balancing in Tor. *Proceedings on Privacy Enhancing Technologies*, 2017(2):74–94, 2017.

[30] Aaron Johnson, Chris Wacek, Rob Jansen, Micah Sherr, and Paul Syverson. Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 337–348, 2013.

[31] Josh Karlin, Daniel Ellard, Alden W. Jackson, Christine E. Jones, Greg Lauer, David P. Mankins, and W. Timothy Strayer. Decoy Routing: Toward Unblockable Internet Communication. In *USENIX Workshop on Free and Open Communications on the Internet*, 2011.

[32] Katharina Kohls, Kai Jansen, David Rupprecht, Thorsten Holz, and Christina Potter. On the Challenges of Geographical Avoidance for Tor. In *Network and Distributed System Security Symposium*, 2019.

[33] Chelsea H. Komlo, Nick Mathewson, and Ian Goldberg. Walking Onions: Scaling Anonymity Networks while Protecting Users. In *USENIX Security Symposium*, pages 1003–1020, 2020.

[34] Tancrède Lepoint and Mehdi Tibouchi. Cryptanalysis of a (Somewhat) Additively Homomorphic Encryption Scheme Used in PIR. In *International Conference on Financial Cryptography and Data Security*, pages 184–193, 2015.

[35] Zhihao Li, Stephen Herwig, and Dave Levin. DeTor: Provably Avoiding Geographic Regions in Tor. In *USENIX Security Symposium*, pages 343–359, 2017.

[36] Angelique Faye Loe and Elizabeth Anne Quaglia. You Shall Not Join: A Measurement Study of Cryptocurrency Peer-to-Peer Bootstrapping Techniques. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 2231–2247, 2019.

[37] Akshaya Mani, T Wilson-Brown, Rob Jansen, Aaron Johnson, and Micah Sherr. Understanding Tor Usage with Privacy-Preserving Measurement. In *Internet Measurement Conference*, pages 175–187, 2018.

[38] Jon McLachlan, Andrew Tran, and Nicholas Hopper. Scalable Onion Routing with Torsk. In *ACM conference on Computer and communications security*, pages 590–599, 2009.

[39] Mohsen Minaei, Pedro Moreno-Sanchez, and Aniket Kate. MoneyMorph: Censorship Resistant Rendezvous using Permissionless Cryptocurrencies. *Proceedings on Privacy Enhancing Technologies*, 2020(3):404–424, 2020.

[40] Michael Mirkin, Yan Ji, Jonathan Pang, Ariah Klages-Mundt, Ittay Eyal, and Ari Juels. BDoS: Blockchain Denial-of-Service. In *ACM SIGSAC conference on Computer and Communications Security*, pages 601–619, 2020.

[41] Prateek Mittal and Nikita Borisov. Shadowwalker: peer-to-peer anonymous communication using redundant structured topologies. In *Proceedings of the 16th ACM conference on Computer and communications security*, pages 161–172, 2009.

[42] Prateek Mittal and Nikita Borisov. Information Leaks in Structured Peer-to-Peer Anonymous Communication Systems. *ACM Transactions on Information and System Security*, 15(1):1–28, 2012.

[43] Prateek Mittal, Femi Olumofin, Carmela Troncoso, Nikita Borisov, and Ian Goldberg. PIR-Tor: Scalable Anonymous Communication Using Private Information Retrieval. In *USENIX Security Symposium*, 2011.

[44] Milad Nasr, Alireza Bahramali, and Amir Houmansadr. DeepCorr: Strong Flow Correlation Attacks on Tor Using Deep Learning. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 1962–1976, 2018.

[45] Andriy Panchenko, Stefan Richter, and Arne Rache. Nisan: network information service for anonymization networks. In *Proceedings of the 16th ACM conference on Computer and communications security*, pages 141–150, 2009.

[46] Andreas Pfitzmann and Marit Hansen. A terminology for talking about privacy by data minimization: Anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management, 2010.

[47] Joseph Poon and Thaddeus Dryja. The bitcoin lightning network: Scalable off-chain instant payments, 2016.

[48] QuikNode. Ethereum api. https://www.quicknode.com/chains/eth, 2022.

[49] Ruben Recabarren and Bogdan Carbunar. Tithonus: A Bitcoin Based Censorship Resilient System. *Proceedings on Privacy Enhancing Technologies*, 2019(1):68–86, 2019.

[50] Michael G Reed, Paul F Syverson, and David M Goldschlag. Anonymous connections and onion routing. *IEEE Journal on Selected areas in Communications*, 16(4):482–494, 1998.

[51] Michael K Reiter and Aviel D Rubin. Crowds: Anonymity for web transactions. *ACM transactions on information and system security (TISSEC)*, 1(1):66–92, 1998.

[52] Marc Rennhard and Bernhard Plattner. Introducing morphmix: Peer-to-peer based anonymous internet usage with collusion detection. In *Proceedings of the 2002 ACM Workshop on Privacy in the Electronic Society*, pages 91–102, 2002.

[53] Rob Jansen. New Tor Denial of Service Attacks and Defenses. https://blog.torproject.org/new-tor-denial-service-attacks-and-defenses, 2014.

[54] Sajin Sasy and Ian Goldberg. ConsenSGX: Scaling Anonymous Communications Networks with Trusted Execution Environments. *Proceedings on Privacy Enhancing Technologies*, 2019(3):331–349, 2019.

[55] Fatemeh Shirazi, Milivoj Simeonovski, Muhammad Rizwan Asghar, Michael Backes, and Claudia Diaz. A survey on routing in anonymous communication protocols. *ACM Computing Surveys (CSUR)*, 51(3):1–39, 2018.

[56] Atul Singh et al. Eclipse attacks on overlay networks: Threats and defenses. In *In IEEE INFOCOM*. Citeseer, 2006.

[57] Parisa Tabriz and Nikita Borisov. Breaking the collusion detection mechanism of morphmix. In *International Workshop on Privacy Enhancing Technologies*, pages 368–383. Springer, 2006.

[58] The Tor Project. Tor Directory Protocol Specification, version 3. https://gitweb.torproject.org/torspec.git/tree/dir-spec.txt.

[59] The Tor Project. Tor Metrics. https://metrics.torproject.org/.

[60] The Tor Project. Tor at the Heart: Bridges and Pluggable Transports. https://blog.torproject.org/tor-heart-bridges-and-pluggable-transports, 2016.

[61] Qiyan Wang, Prateek Mittal, and Nikita Borisov. In search of an anonymous and secure lookup: attacks on structured peer-to-peer anonymous communication systems. In *Proceedings of the 17th ACM conference on Computer and communications security*, pages 308–318, 2010.

[62] Gavin Wood. Ethereum: A Secure Decentralised Generalised Transaction Ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.

[63] Quanqing Xu, Zhiwen Song, Rick Siow Mong Goh, and Yongjun Li. Building an Ethereum and IPFS-based Decentralized Social Network System. In *International Conference on Parallel and Distributed Systems*, pages 1–6, 2018.

[64] Lin Zhang, Brian Lee, Yuhang Ye, and Yuansong Qiao. Ethereum transaction performance evaluation using testnets. In *European Conference on Parallel Processing*, 2019.